

Byte-Pair Encoding

How a tokenizer learns its vocabulary, step by step · video 3:04

This lesson answers one question: where does a tokenizer's vocabulary come from? **Byte-pair encoding** starts from the 256 possible **bytes** and repeatedly **merges the most frequent neighbouring pair** into a new token. Trained from scratch on Shakespeare, it learns sensible pieces, but also glues names, punctuation and new lines together, until **pre-tokenization** restricts merges to inside words, as GPT-2 does.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Start from bytes	▶ 0:08–0:41	2	☐
2. Merge the most frequent pair, again and again	▶ 0:41–1:28	3	☐
3. Pre-tokenization: merge only inside words	▶ 1:28–2:04	2	☐
4. GPT-2, and how big a vocabulary should be	▶ 2:04–3:02	2	☐

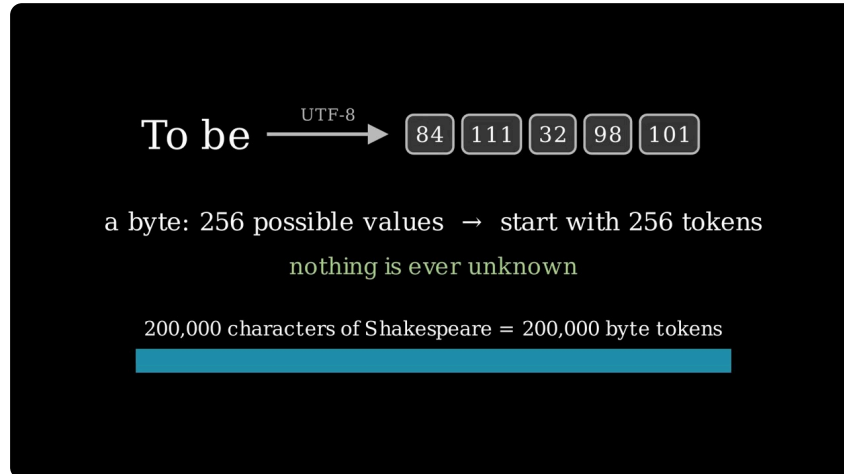
Before you start: How LLMs Work episode 2 (tokens). Code: `code/d01_bpe/bpe.py` (pure Python; `transformers` only for the GPT-2 comparison).

CONCEPT 1 OF 4

Start from bytes

▶ Video 0:08-0:41

Any text, in any language, can be written as **bytes** (with UTF-8), and a byte has only **256** possible values. So BPE begins with a vocabulary of exactly 256 tokens: nothing is ever unknown. But sequences are long: 200,000 characters of Shakespeare are 200,000 tokens.



▶ 0:39 "To be" as UTF-8 bytes; 256 starting tokens; 200,000 characters = 200,000 byte tokens.

KEY IDEA

Bytes guarantee coverage; merges buy back shorter sequences.

Check 1: Start from bytes

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

"To be" becomes the bytes 84, 111, 32, 98, 101. Which byte is the space?

1.2 TRUE OR FALSE

"A byte-level tokenizer can still meet a word it has no tokens for."

True ☐ False ☐

CONCEPT 2 OF 4

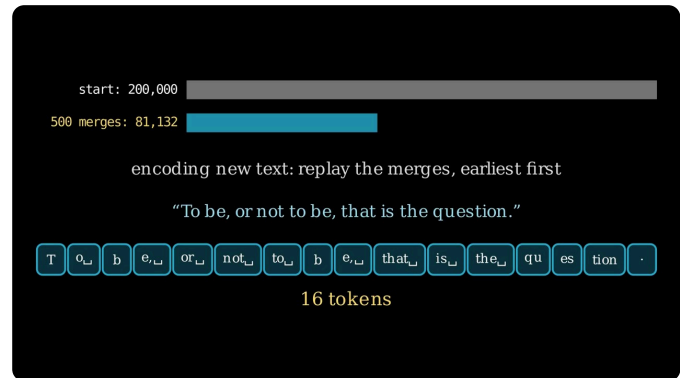
Merge the most frequent pair, again and again

▶ Video 0:41-1:28

The algorithm: count every pair of neighbouring tokens, take the most frequent pair, give it a new token number, replace it everywhere, and repeat. On Tiny Shakespeare, merge 1 is “**e**” + **space**, seen 5,249 times; it becomes token 256. Then t + h, t + space, s + space, o + u. Each merge makes the text a little shorter: 200,000 → 194,751 → 190,686 ...

merge	pair	new token	seen	text length
1	e + 	e_	5,249	194,751
2	t + h	th	4,065	190,686
3	t + 	t_	2,936	187,750
4	s + 	s_	2,797	184,953
5	o + u	ou	2,529	182,424

“e_” becomes token 256, the first new token



▶ 1:09 The real first merges on Tiny Shakespeare: e + space, t + h, t + space, s + space, o + u.

▶ 1:27 After 500 merges: 81,132 tokens; the Hamlet line is 16 tokens.

After 500 merges the text is **81,132** tokens. To encode new text, replay the merges in the order they were learned: “*To be, or not to be, that is the question.*” becomes 16 tokens, with “question” split into “qu”, “es”, “tion”.

KEY IDEA

The vocabulary is just the history of merges: frequent sequences in the training text become single tokens.

Check 2: Merge the most frequent pair, again and again

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Run BPE by hand on *aaabdaaabc* (11 characters). After each of the first three merges, how many tokens are left?

2.2 CALCULATION

Merge 1 (“e” + space, seen 5,249 times) shrank the text from 200,000 to 194,751 tokens. By how many tokens?

2.3 PUT IN ORDER

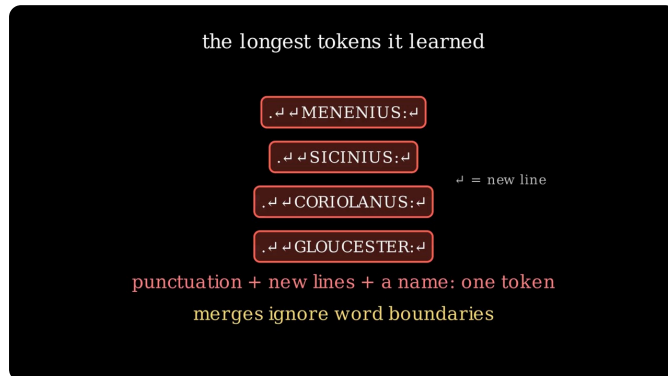
Order the steps of one BPE iteration: *replace everywhere* · *count neighbouring pairs* · *add a new token id* · *pick the most frequent pair*.

CONCEPT 3 OF 4

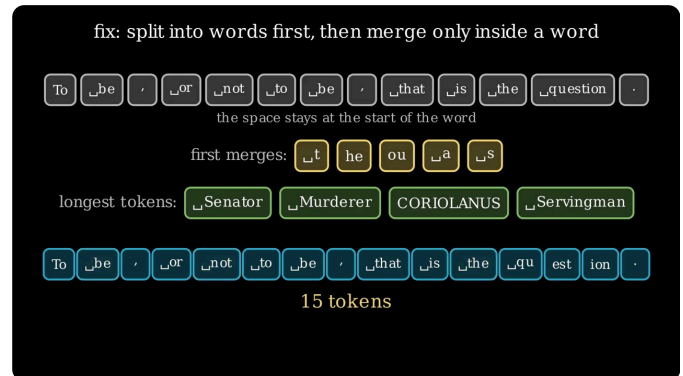
Pre-tokenization: merge only inside words

▶ Video 1:28-2:04

The longest tokens the naive version learned: a period, two new lines, the name *CORIANUS*, a colon and another new line, all glued into one token. The merges ignore word boundaries, so punctuation, spacing and names fuse together.



▶ 1:42 Naive BPE glues punctuation, new lines and a name into one token.



▶ 2:03 Split into words first: tokens like “ $\backslash$ t”, whole words like “ $\backslash$ Senator”; 15 tokens.

The fix: split the text into **words first**, keeping each space at the start of the word, and only merge inside a word. The first merges become “ $\backslash$ t”, “he”, “ou”; the longest tokens are whole words like “ $\backslash$ Senator”; and the Hamlet line becomes 15 tokens: “ $\backslash$ be”, “ $\backslash$ not”, “ $\backslash$ the”, just like a real tokenizer.

KEY IDEA

Pre-tokenization decides which merges are allowed; it shapes the vocabulary as much as the data does.

Check 3: Pre-tokenization: merge only inside words

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

Why is a token like “. $\backslash$ CORIANUS:.” a bad use of the vocabulary?

3.2 MULTIPLE CHOICE

With word pre-splitting, where does the space go?

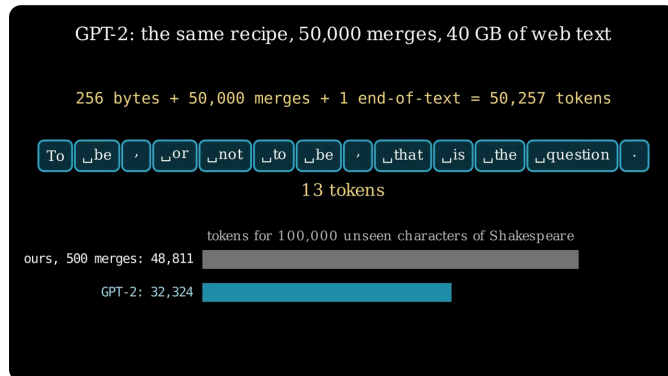
- A. At the end of each word
- B. At the start of each word (“ $\backslash$ be”)
- C. It is deleted
- D. It is always a separate token

CONCEPT 4 OF 4

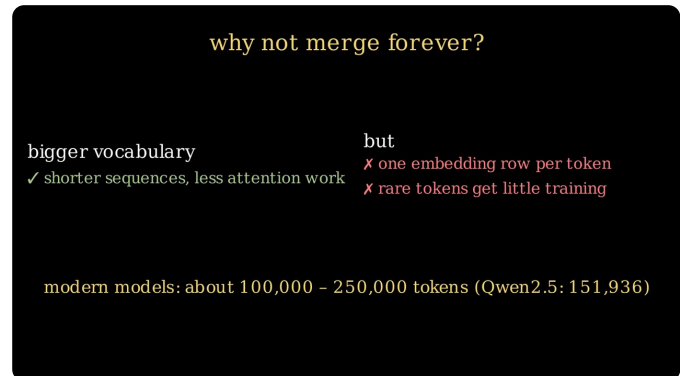
GPT-2, and how big a vocabulary should be

▶ Video 2:04–3:02

GPT-2 uses exactly this recipe: **50,000 merges** learned from 40 GB of web text. Its vocabulary is 256 bytes + 50,000 merges + 1 end-of-text token = **50,257**. The Hamlet line is 13 tokens, with “`␣question`” a single token. On 100,000 characters of Shakespeare it has not seen, GPT-2 needs 32,324 tokens, about a third fewer than our 500-merge tokenizer (48,811).



▶ 2:27 GPT-2: 256 + 50,000 + 1 = 50,257 tokens; the Hamlet line is 13 tokens.



▶ 2:44 A bigger vocabulary: shorter sequences, but more embeddings and rarer tokens.

Why not merge forever? A bigger vocabulary makes sequences **shorter** (less attention work, episode 2 of LLMs in Practice), but every token needs its own **embedding**, and **rare** tokens get little training. Modern models settle on about 100,000–250,000 tokens (Qwen2.5: 151,936).

KEY IDEA

Vocabulary size trades sequence length against embedding size and how well each token is trained.

Check 4: GPT-2, and how big a vocabulary should be

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

GPT-2's embedding vectors have 768 numbers. How many numbers are in its token embedding table?

4.2 HANDS-ON CODE

Try it yourself. In `code/d01_bpe/bpe.py`, set `N_MERGES = 100` and run it. (a) How many tokens is the Hamlet line for the naive tokenizer, and how many for unseen text? (b) Compare with 500 merges.

```
N_MERGES = 100
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Start from bytes · rewatch ▶ 0:08-0:41

1.1 32. In UTF-8 (and ASCII) the space character is byte 32.

1.2 False. Any text can be written with the 256 byte tokens, so the worst case is just more tokens.

2. Merge the most frequent pair, again and again · rewatch ▶ 0:41-1:28

2.1 9, 7, 5. Merge “aa” → [aa]abd[aa]abac (9); merge [aa]+a → (7); merge [[aa]a]+b → (5). (Counting overlapping pairs, “aa” appears 4 times, but only 2 non-overlapping copies can be merged.)

2.2 5,249. Every merged pair turns two tokens into one.

2.3 count neighbouring pairs → pick the most frequent pair → add a new token id → replace everywhere. Then the loop starts again on the shorter sequence.

3. Pre-tokenization: merge only inside words · rewatch ▶ 1:28-2:04

3.1 It only fits one exact pattern in this play; it wastes a vocabulary slot and teaches the model nothing reusable about words or punctuation. Good tokens are pieces that recur in many contexts.

3.2 B. GPT-2-style splitting attaches the space to the following word.

4. GPT-2, and how big a vocabulary should be · rewatch ▶ 2:04-3:02

4.1 38,597,376. $50,257 \times 768 = 38,597,376$: one row per token.

4.2 (a) 21 tokens; 62,593 tokens for the 100,000 unseen characters (1.60 bytes per token). (b) With 500 merges: 16 tokens and 46,714 (2.14 bytes per token). More merges, shorter sequences. Each merge is a new vocabulary entry that can replace a frequent pair.